

! Submission instructions
not followed



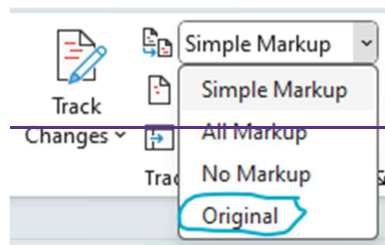
DELETE THIS PAGE BEFORE SUBMISSION

Notes to the Dev Crew

~~This is a template for the **Functional Specification & Solution Blueprint** for your project with explanatory notes included.~~

~~When using this template, the following steps are recommended:~~

- ~~7. Replace all text enclosed in angle brackets (e.g., <Crew Name>) with the correct values. These angle brackets appear either in the body of the document or in headers and footers.~~
- ~~8. To update the Table of Contents, right-click on it and select "Update field" and choose the option—"Update entire table".~~
- ~~9. Go to the Review tab; make sure Display for Review is set to "Original".~~



- ~~13. Before submission of this document, delete this section "Notes to the Dev Crew".~~
- ~~14. Save the document as a pdf file with the following name FSSB_<CrewName>.pdf~~



WRRV301
2026

Functional Specification & Solution Blueprint

April 2026

Commented [N1]((CS1):

This document will describe the functions that the system under development must perform in order to meet the user's objectives. The document typically describes the product's intended abilities, its appearance and characteristics and how users will interact with the product.

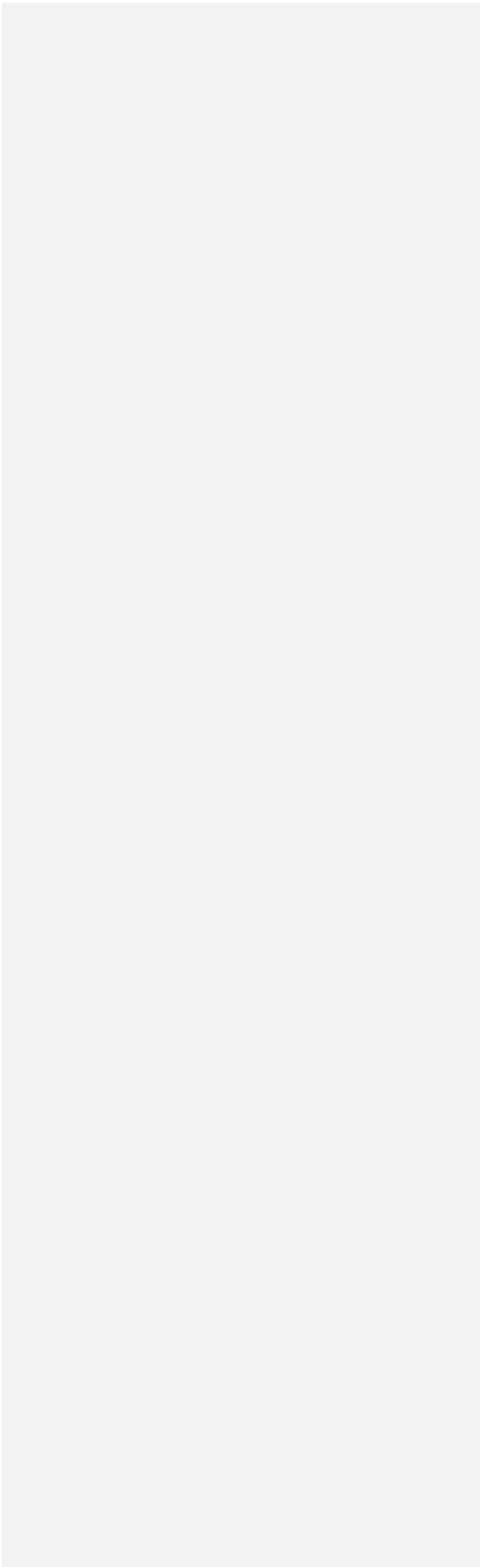
<BiDE – Booking & Instructor Dispatch Engine>

<Essential Minds>

67% <Lethabo Mathole>
69% <Hlomani Lloyd Mashimbye>
70% <Awonke Gana>

grel

Commented [JN2]: If you are 3 in the Dev Crew, then delete the additional <name surname>



1	SYSTEM OVERVIEW	5
1.1	Problem Description	5
1.2	Business Rules / Assumptions	5
1.3	System Constraints	5
2	FUNCTIONAL REQUIREMENTS	7
2.1	Initial Use Case Model	7
2.2	Final Use Case Model	8
2.3	Use Case Glossary and Responsibilities	9
2.4	Use Case Narratives & UI Designs	11
2.4.1	Team Designs	11
2.4.1.1	Team User Interface Guidelines	11
2.4.1.2	Login / Logout Feature	11
2.4.1.3	Dashboard / Landing Page / Main Page	12
2.4.2	Individual Crew Member Use Case Narratives & UI Designs	16
2.4.2.1	Designed by <Hlomani Mashimbye>	16
2.4.2.2	Designed by <Hlomani Mashimbye>	17
2.4.2.3	Designed by <Hlomani Mashimbye>	19
2.4.2.4	Designed by <Hlomani Mashimbye>	21
2.4.2.5	Designed by <Hlomani Mashimbye>	21
2.4.2.6	Designed by <Lethabo Mathole>	22
2.4.2.7	Designed by <Lethabo Mathole>	24
2.4.2.8	Designed by <Lethabo Mathole>	24
2.4.2.9	Designed by <Awonke Gana>	26
2.4.2.10	Designed by <Lethabo Mathole>	27
2.4.2.11	Designed by <Awonke Gana>	28
2.4.2.12	Designed by <Awonke Gana>	30
2.4.2.13	Designed by <Awonke Gana>	31
2.4.2.14	Designed by <Awonke Gana>	32
2.4.2.15	Designed by <Lethabo Mathole>	33
2.4.2.16	Designed by <Hlomani Mashimbye>	35
2.4.2.17	Designed by <Hlomani Mashimbye>	36
2.4.2.18	Designed by <View Payments>	38
2.4.2.19	Designed by <Awonke Gana>	39
2.4.2.20	Designed by <Awonke Gana>	40
3	DATA REQUIREMENTS	41
3.1	List of data and attributes	41
3.2	Domain Class Diagram	42
3.3	Draft Implementation Ready Class Diagram	43
3.4	Implemented database structure	45
4	TECHNICAL REQUIREMENTS	46
4.1	System Environment	46
4.1.1.1	Operating Environment	46
4.1.1.2	Development Tools and Technologies	46

4.1.1.3 Additional Components (DLLs / Libraries)46

4.1.1.4 Recommended Screen Resolution.....46

4.1.1.5 Other Requirements.....46

4.2 Start-up Instructions47

4.3 Logins & Passwords47

TABLE OF CONTENTS

1 SYSTEM OVERVIEW7

1.1 Problem Description7

1.2 Business Rules / Assumptions7

1.3	System Constraints	7
2	FUNCTIONAL REQUIREMENTS	9
2.1	Initial Use Case Model.....	9
2.2	Final Use Case Model	10
2.3	Use Case Glossary and Responsibilities	11
2.4	Use Case Narratives & UI Designs	12
2.4.1	Team Designs.....	12
2.4.1.1	Team User Interface Guidelines	12
2.4.1.2	Login / Logout Feature.....	12
2.4.1.3	Dashboard / Landing Page / Main Page.....	13
2.4.2	Individual Crew Member Use Case Narratives & UI Designs	17
2.4.2.1	Designed by <Hlomani Mashimbye>	17
2.4.2.2	Designed by <Hlomani Mashimbye>	18
2.4.2.3	Designed by <Hlomani Mashimbye>	20
2.4.2.4	Designed by <Hlomani Mashimbye>	22
2.4.2.5	Designed by <Hlomani Mashimbye>	22
2.4.2.6	Designed by <Lethabo Mathole>	23
2.4.2.7	Designed by <Lethabo Mathole>	25
2.4.2.8	Designed by <Lethabo Mathole>	25
2.4.2.9	Designed by <Awonke Gana>	27
2.4.2.10	Designed by <Lethabo Mathole>	28
2.4.2.11	Designed by <Awonke Gana>	29
2.4.2.12	Designed by <Awonke Gana>	31
2.4.2.13	Designed by <Awonke Gana>	32
2.4.2.14	Designed by <Awonke Gana>	33
2.4.2.15	Designed by <Lethabo Mathole>	34
2.4.2.16	Designed by <Hlomani Mashimbye>	36
2.4.2.17	Designed by <Hlomani Mashimbye>	37
2.4.2.18	Designed by <View Payments>	39
2.4.2.19	Designed by <Awonke Gana>	40
2.4.2.20	Designed by <Awonke Gana>	41
3	DATA REQUIREMENTS.....	42
3.1	List of data and attributes	42
3.2	Domain Class Diagram.....	Error! Bookmark not defined.
3.3	Draft Implementation Ready Class Diagram	44
3.4	Implemented database structure	46
4	TECHNICAL REQUIREMENTS.....	47
4.1	System Environment	47
4.1.1.1	Operating Environment.....	47
4.1.1.2	Development Tools and Technologies	47
4.1.1.3	Additional Components (DLLs / Libraries)	47
4.1.1.4	Recommended Screen Resolution	47
4.1.1.5	Other Requirements.....	47
4.2	Start-up Instructions	48
4.3	Logins & Passwords	48

1	SYSTEM OVERVIEW	5
1.1	Problem Description	5
1.2	Business Rules / Assumptions	5
1.3	System Constraints	5
2	FUNCTIONAL REQUIREMENTS	7
2.1	Initial Use Case Model	7
2.2	Final Use Case Model	8
2.3	Use Case Glossary and Responsibilities	9
2.4	Use Case Narratives & UI Designs	11
2.4.1	Team Designs	11
2.4.1.1	Team User Interface Guidelines	11
2.4.1.2	Login / Logout Feature	11
2.4.1.3	Dashboard / Landing Page / Main Page	12
2.4.2	Individual Crew Member Use Case Narratives & UI Designs	16
2.4.2.1	Designed by <Hlomani Mashimbye>	16
2.4.2.2	Designed by <Hlomani Mashimbye>	17
2.4.2.3	Designed by <Hlomani Mashimbye>	19
2.4.2.4	Designed by <Hlomani Mashimbye>	21
2.4.2.5	Designed by <Hlomani Mashimbye>	21
2.4.2.6	Designed by <Lethabo Mathole>	22
2.4.2.7	Designed by <Lethabo Mathole>	24
2.4.2.8	Designed by <Lethabo Mathole>	24
2.4.2.9	Designed by <Awonke Gana>	26
2.4.2.10	Designed by <Lethabo Mathole>	27
2.4.2.11	Designed by <Awonke Gana>	28
2.4.2.12	Designed by <Awonke Gana>	30
2.4.2.13	Designed by <Awonke Gana>	31
2.4.2.14	Designed by <Awonke Gana>	32
2.4.2.15	Designed by <Lethabo Mathole>	33
2.4.2.16	Designed by <Hlomani Mashimbye>	35
2.4.2.17	Designed by <Hlomani Mashimbye>	36
2.4.2.18	Designed by <View Payments>	38
2.4.2.19	Designed by <Awonke Gana>	39
2.4.2.20	Designed by <Awonke Gana>	40
3	DATA REQUIREMENTS	41
3.1	List of data and attributes	41
3.2	Domain Class Diagram	42
3.3	Draft Implementation Ready Class Diagram	43
3.4	Implemented database structure	45
4	TECHNICAL REQUIREMENTS	46
4.1	System Environment	46
4.1.1.1	Operating Environment	46
4.1.1.2	Development Tools and Technologies	46
4.1.1.3	Additional Components (DLLs / Libraries)	46
4.1.1.4	Recommended Screen Resolution	46

4.1.1.5—Other Requirements	46
4.2—Start-up Instructions	47
4.3—Logins & Passwords	47
1—SYSTEM OVERVIEW	4
1.1—Problem Description	4
1.2—Business Rules / Assumptions	4
1.3—System Constraints	4
2—FUNCTIONAL REQUIREMENTS	5
2.1—Initial Use Case Model	5
2.2—Final Use Case Model	6
2.3—Use Case Glossary and Responsibilities	7
2.4—Use Case Narratives & UI Designs	8
2.4.1—Team Designs	8
2.4.1.1—Team User Interface Guidelines	8
2.4.1.2—Login / Logout Feature	8
2.4.1.3—Dashboard / Landing Page / Main Page	8
2.4.2—Individual Crew Member Use Case Narratives & UI Designs	9
2.4.2.1—Designed by <Insert crew member name surname>	9
2.4.2.2—Designed by <Insert dev crew member name surname>	10
2.4.2.3—Designed by <Insert dev crew member name surname>	11
2.4.2.4—Designed by <Insert dev crew member name surname>	12
4—DATA REQUIREMENTS	13
4.1—List of data and attributes	13
4.2—Domain Class Diagram	14
4.3—Draft Implementation Ready Class Diagram	15
4.4—Implemented database structure	16
5—TECHNICAL REQUIREMENTS	17
5.1—System Environment	17
5.2—Start-up Instructions	17
5.3—Logins & Passwords	17

1 SYSTEM OVERVIEW

1.1 Problem Description



Many individuals who want to learn how to drive experience difficulties in finding reliable, affordable, and accessible driving instructors. Currently, there is no centralized platform that allows learners to easily search for instructors based on location, price, availability, and experience. As a result, learners often rely on informal methods such as word-of-mouth, social media, or messaging platforms, which are inefficient and lack transparency.

At the same time, experienced driving instructors who are willing to offer lessons lack a structured platform to list their services, manage bookings, or track student progress. This leads to missed opportunities, poor scheduling, and ineffective communication between students and instructors.

Therefore, the objective of this project is to develop a Bidirectional Driving Lesson Marketplace System that connects Individuals who want to learn how to drive (Students) and Experienced drivers who want to offer driving lessons (Instructors). The system acts as a two-sided platform where both students and instructors interact through searching, reviewing, and flexible scheduling for both single and multi-session bookings.

1.2 Business Rules / Assumptions

- ✓ 1. A student must register and have a verified account before making a booking request.
- 2. An instructor must apply and be approved by an administrator before appearing in search results.
- 3. A booking request is only confirmed after payment proof has been uploaded by the student and verified by the instructor.
- 4. A student may only leave a review for a lesson that has been completed.
- 5. A booking may be cancelled or rescheduled subject to system validation.
- 6. An instructor sets up their own availability; bookings can only be requested within those available slots.
- 7. Payment is handled externally (EFT/cash); only proof of payment is uploaded to the system.
- 7. Each booking is linked to exactly one lesson offering.
- 8. The administrator has full access to manage users, view payments, monitor bookings, and generate reports.

1.3 System Constraints

The development of this system is subject to several constraints that may influence the design and implementation:

- ✓ • The system will be developed as a **web-based application only** and will not include mobile or desktop versions.
- Payment functionality will be **simulated**, meaning no real payment gateway or financial transactions will be integrated.
- The system will be developed using the technologies specified in the module (e.g., HTML, CSS, JavaScript, and ASP.NET/MVC), which may limit the use of advanced frameworks or tools.
- The project must be completed within a limited timeframe as defined by the academic schedule.
- The system will operate in a **controlled environment** and will not include real-world verification of users (e.g., instructor certification validation).
- Data used in the system may be **sample or test data**, and not actual real-world data.
- The system will support core functionalities such as booking, lesson progress tracking, and reviews, but may not include advanced features such as real-time notifications or integrated messaging.

Commented [JN3]: Constraints are all conditions beyond the control of the project that may in any way limit the development options.

Example:
If it is an online order system, payments will be simulated

2.2 Final Use Case Model

[Replace this text with the Use Case diagram – this must be drawn in Visio.]

Commented [JN6]: ONLY TO BE ADDED AT END OF YEAR

This will be an updated use case diagram based on feedback from the FSSB document assessment AS WELL AS any additions / deletions made during Implementation.

VERY NB:
Please include Use Case ID on the use case diagram as well

2.3 Use Case Glossary and Responsibilities

* IDs on model does not all match
lbs here

Allocated to Crew Member: <Hlomani Mashimbye>	
Use Case Id	Use Case Name
UC02_SI ✓	Register Account
UC02_SI ✓	Edit Profile
UC03_S ✓	View Lesson Progress
UC04_S ✓	Search Instructor
UC05_S ✓	View Instructor Profile
UC16_A ✓	Review Instructor Application
UC17_A ✓	Manage Users

Allocated to Crew Member: <Lethabo Mathole>	
Use Case Id	Use Case Name
UC06_S ✓	Make Booking Request
UC15_I ✓	Verify Payment
UC08_S ✓	Upload Proof of Payment
UC10_S ✓	Leave Review
UC07_S ✓	View Booking Status
UC13_I ✓	View Booking Requests
UC18_A ✓	View Payments

Allocated to Crew Member: Awonke Gana >	
Use Case Id	Use Case Name
UC11_SI ✓	Manage Bookings
UC14_I ✓	Respond to Booking Request
UC10_I ✓	Provide Lesson Feedback
UC11_I ✓	Manage Availability
UC12_I ✓	Manage Lesson Offerings
UC09_I ✓	View Completed Lessons
UC19_A ✓	Monitor Bookings

narrative + design missing

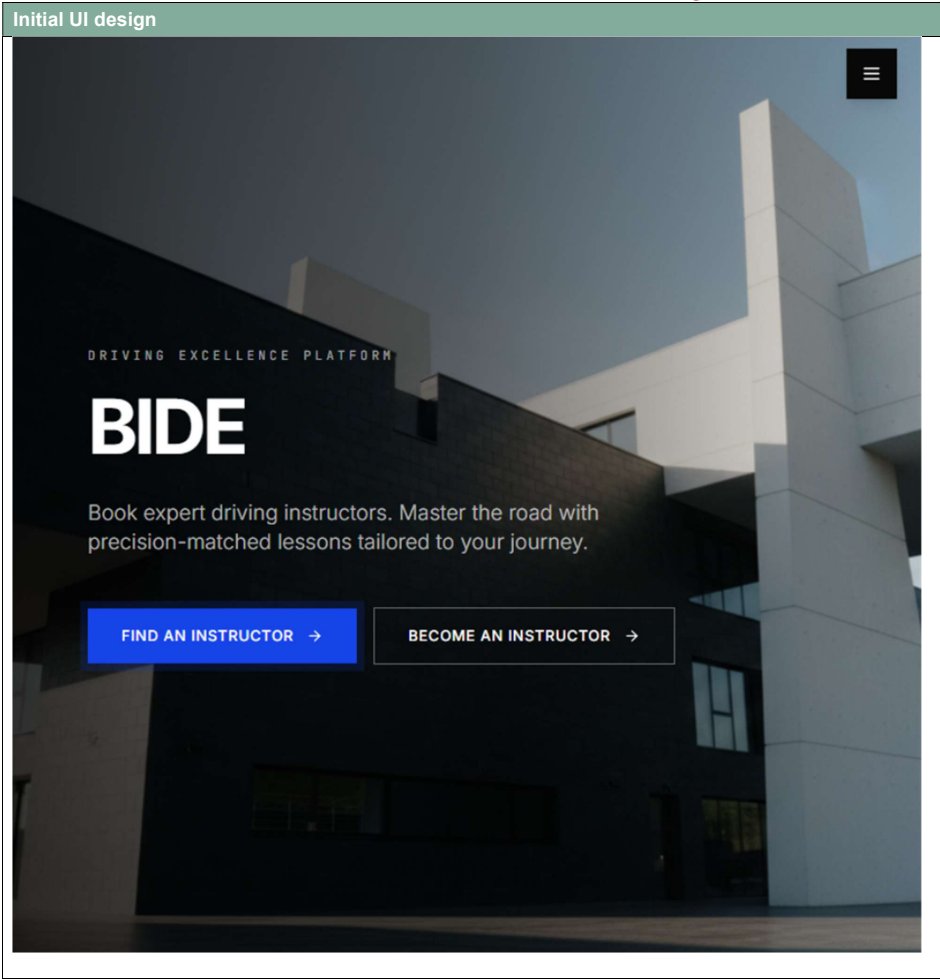
2.4 Use Case Narratives & UI Designs

2.4.1 Team Designs

2.4.1.1 Team User Interface Guidelines

2.4.1.2 Login / Logout Feature

the 2 UIs below are the Landing pages, show Login



Commented [JN7]:
You may do the design in your chosen development environment

OR

you may make use of the any UI prototyping tools to assist you with your designs:

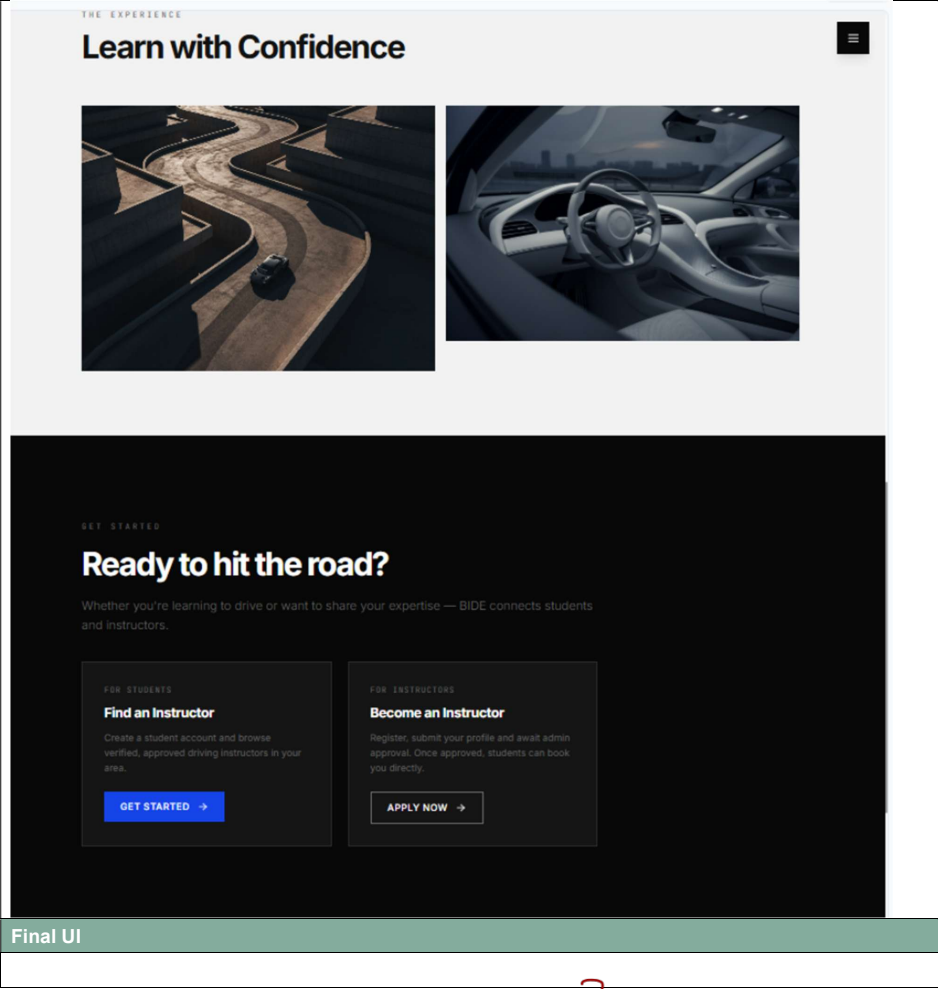
Downloadable Tools
Balsamiq (<https://balsamiq.com>)
Justinmind (<http://www.justinmind.com>)
Wireframesketcher (<http://wireframesketcher.com>)

Online Tools
Fluidui (www.fluidui.com)
Mockflow (www.mockflow.com)
Canava (www.canva.com)
Figma (www.figma.com)

Include screen snips of the design for each use case completed by each crew member. Some use cases might be designed on one UI or split over multiple designs.

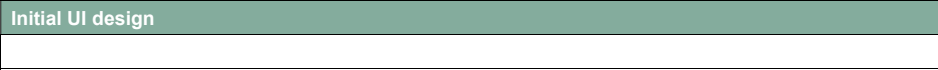
You may provide annotated notes to your designs

NO HAND DRAWN DESIGN / WIREFRAMES ARE PERMITTED



2 UIs above?

2.4.1.3 Dashboard / Landing Page / Main Page



SEARCH

Find Your Instructor

Q Search by name or location...

All Types



lemonstacks

Gqeberha

★ 4.0 (1) 12 yrs



hlomanilloyd

north end

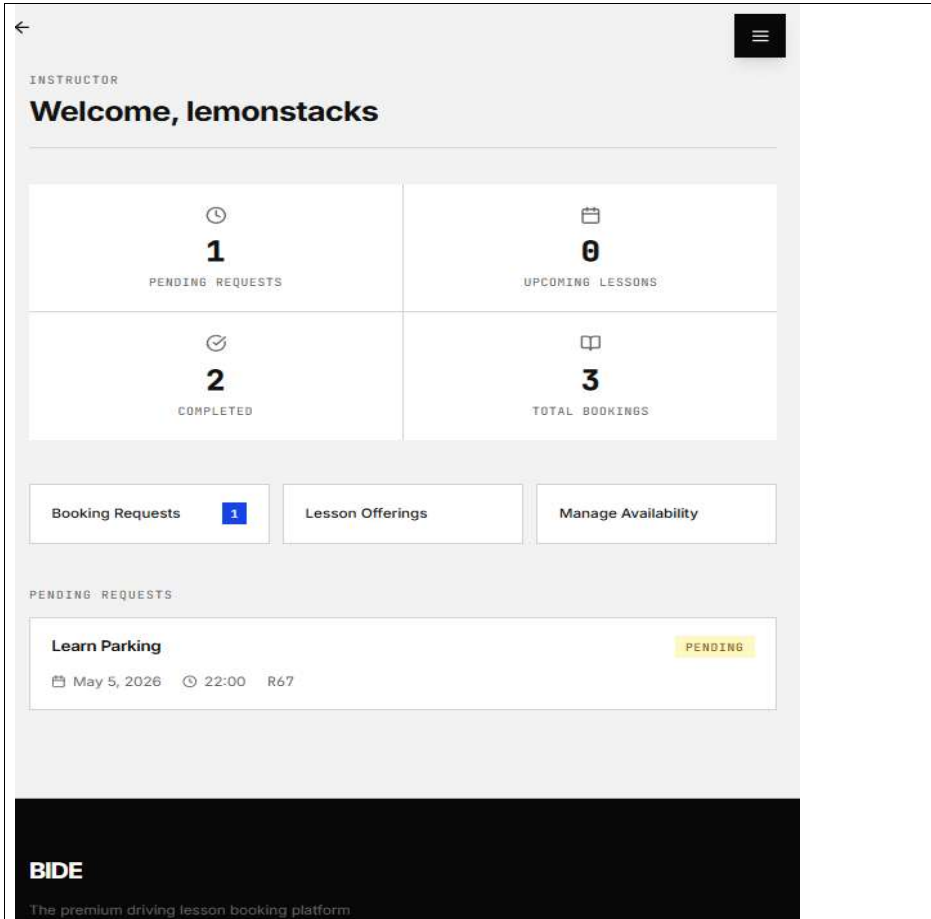
25 yrs

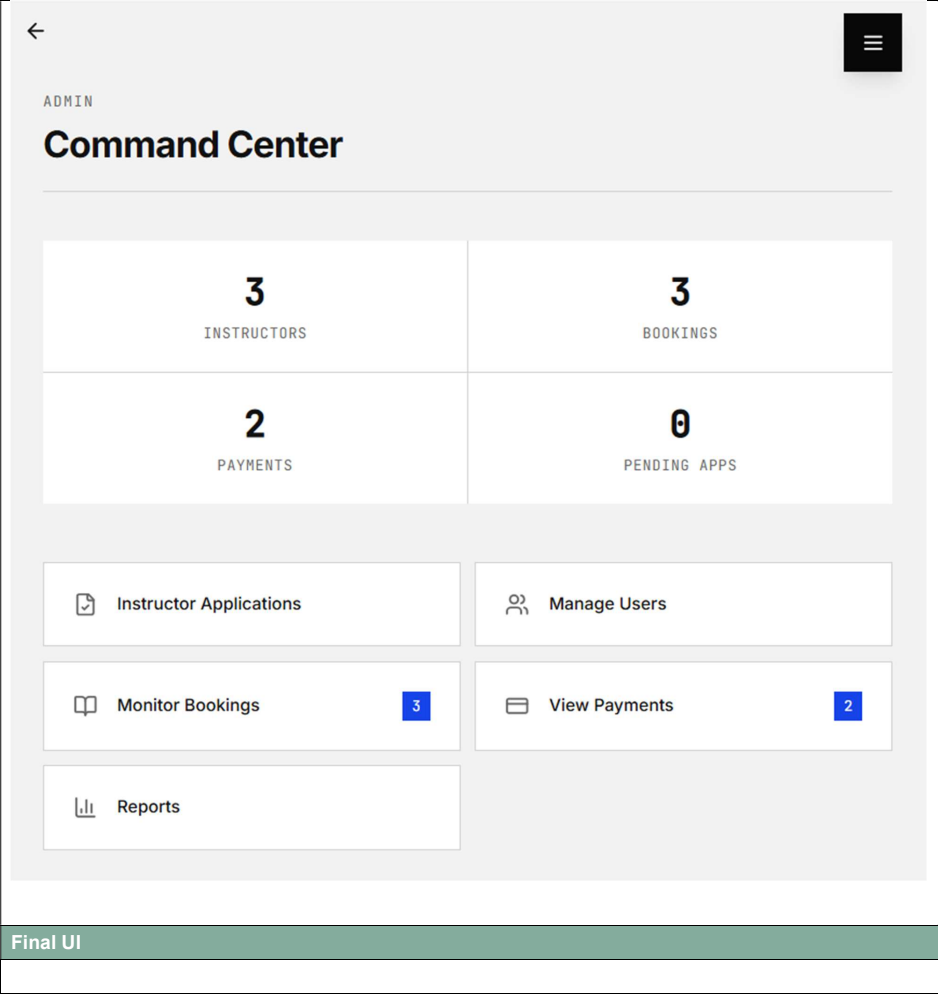
D

Daid

PE

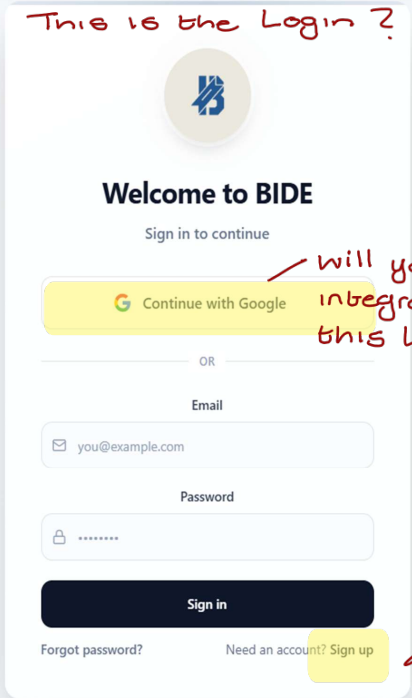
- are these instructors
in order of rating or
proximity to your location?





2.4.2 Individual Crew Member Use Case Narratives & UI Designs

2.4.2.1 Designed by <Hlomani Mashimbye>

Use Case ID	Use Case Name
UC01_SI	Register Account
Actors	
Student, Instructor	
Pre-Conditions	Users are not registered in the system.
Triggers	User selects register option <i>Sign Up</i>
Post-Conditions	Account is created and stored
Basic Flow of Events / Main Success Scenario	
The actor selects the option to register an account. - The system displays the registration form. - The actor enters personal details. - The system validates the information provided. - The system creates the account. - The system displays a confirmation message.	
Initial UI Design	
	
Final Implemented UI Design	

Commented [JN8]: A simple (or high-level) use case narrative is a brief description of how a user interacts with a system to achieve a specific goal. It explains the main interaction between the user and the system without going into detailed steps or alternative scenarios.

It focuses on **what happens at a general level**, rather than describing every action or exception.

Commented [JN9]: Repeat tables below for every use case on the use case model
Ensure numbering in each table for the flow of events start at 1 for each use case

Commented [JN10]: Use case ID must correspond with Use Case model

✓ The use case has a **unique identifier** (e.g., UC01, UC02)

✓ The numbering is **consistent across all use cases** in the system

✓ The ID is used to **reference the use case elsewhere in the documentation**

Commented [JN11]: ✓ The name clearly describes the user action

✓ The name begins with a verb (e.g., *Submit Project Preferences, View Reports, Register User*)

✓ The name represents one specific task or goal ... [1]

Commented [JN12]: ✓ The primary actor is clearly identified

✓ The actor is an **external user or system interacting with the system** ... [2]

Commented [Janine13]: Pre-conditions describe **what must already be true before the use case can begin**.

✓ The pre-condition describes the **required system state**

✓ It explains **what must already exist or be complete** ... [3]

Commented [Janine14]: Triggers describe **the event that starts the use case**.

✓ The trigger clearly explains **what action initiates the use case** ... [4]

Commented [Janine15]: Post-conditions describe **what must be true after the use case has completed successfully**.

✓ The outcome of the use case is clearly stated ... [5]

Commented [Janine16]: This section describes the **main interaction between the actor and the system**.

✓ The flow contains **5-7 logical steps** ... [6]

Commented [JN17]: Include a screen design that will help the reader visualize the behavior of the use case. Annotate / provide explanatory notes where necessary.

This section represents the **early concept or prototype of the interface**. ... [7]

Commented [JN18]: This section shows the actual implemented interface in the system.

✓ A **screenshot or final interface design** is included ... [8]

2.4.2.2 Designed by <Hlomani Mashimbye>

Use Case ID	Use Case Name
UC02_SI	Edit Profile
Actors	
Student, Instructor	
Pre-Conditions	User has an account , Logged in
Triggers	User selects edit profile from ≡ menu
Post-Conditions	Profile information is updated
Basic Flow of Events / Main Success Scenario	
1. The actor selects the option to edit their profile. 2. The system displays current profile information. 3. The actor updates the required details. 4. The system validates the updated information. 5. The system saves the changes. 6. The system displays the updated profile.	
Initial UI Design	

Commented [Janine19]:
Repeat tables below for every use case
Ensure numbering in each table for the flow of events start at 1 for each use case

Alt
- user press Delete Profile
- system

INSTRUCTOR

Instructor Profile

APPLICATION STATUS

APPROVED

You are an approved instructor. Students can find and book you.

DELETE PROFILE

PROFILE PHOTO

not an attribute

Choose File

No file chosen

FULL NAME *

lemonstacks

BIO

just stacks and queues

LOCATION / AREA

Suburb

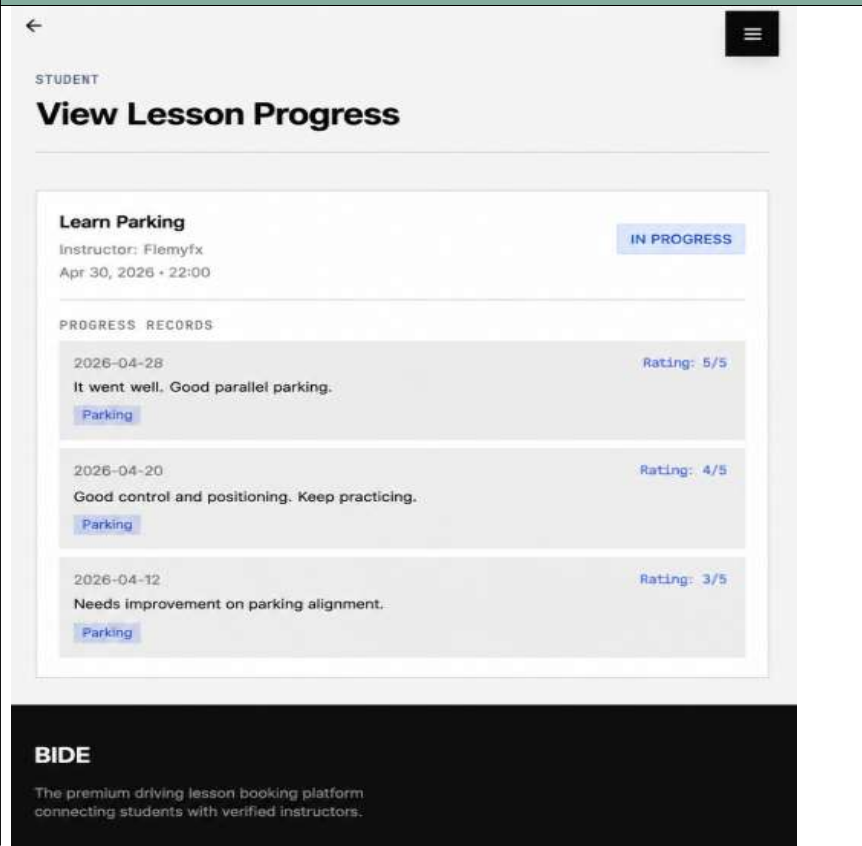
YEARS OF EXPERIENCE

12

INSTRUCTOR LICENSE NUMBER

123456789

UPDATE PROFILE

Actors	
Student	
Pre-Conditions	User has an account Student has at least one booking with recorded lesson progress
Triggers	Student selects View Lesson Progress <i>from</i> 
Post-Conditions	Lesson progress information is displayed
Basic Flow of Events / Main Success Scenario	
The actor selects the option to view lesson progress. - The system retrieves the student's booking records. - The system retrieves associated lesson progress records. - The system displays lesson details (lesson name, instructor, date, status). - The system displays all progress records including feedback, rating, and skills covered. - The system highlights the current lesson status (e.g., In Progress or Completed).	
Initial UI Design	
 The mockup shows a mobile app interface for a student. At the top, there's a back arrow and a hamburger menu icon. Below the 'STUDENT' header, the title 'View Lesson Progress' is displayed. The main content area features a lesson card for 'Learn Parking' by instructor 'Flemyfx', dated 'Apr 30, 2026 • 22:00'. The status 'IN PROGRESS' is shown in a blue box. Below this, a section titled 'PROGRESS RECORDS' lists three entries with dates, descriptions, ratings, and 'Parking' tags. A footer section for 'BIDE' describes it as a premium driving lesson booking platform.	
Final Implemented UI Design	

2.4.2.4 Designed by <Hlomani Mashimbye>

Use Case ID	Use Case Name
UC04_S	Search Instructor
Actors	
Student	
Pre-Conditions	Student account
Triggers	Student enters search criteria on.....
Post-Conditions	Matching instructors are displayed
Basic Flow of Events / Main Success Scenario	
1. The actor enters search criteria. 2. The system processes the request. 3. The system retrieves matching instructors. 4. The system displays search results. , 5. Student select Profile to view 6. System calls UC05_S	
Initial UI Design	
Final Implemented UI Design	

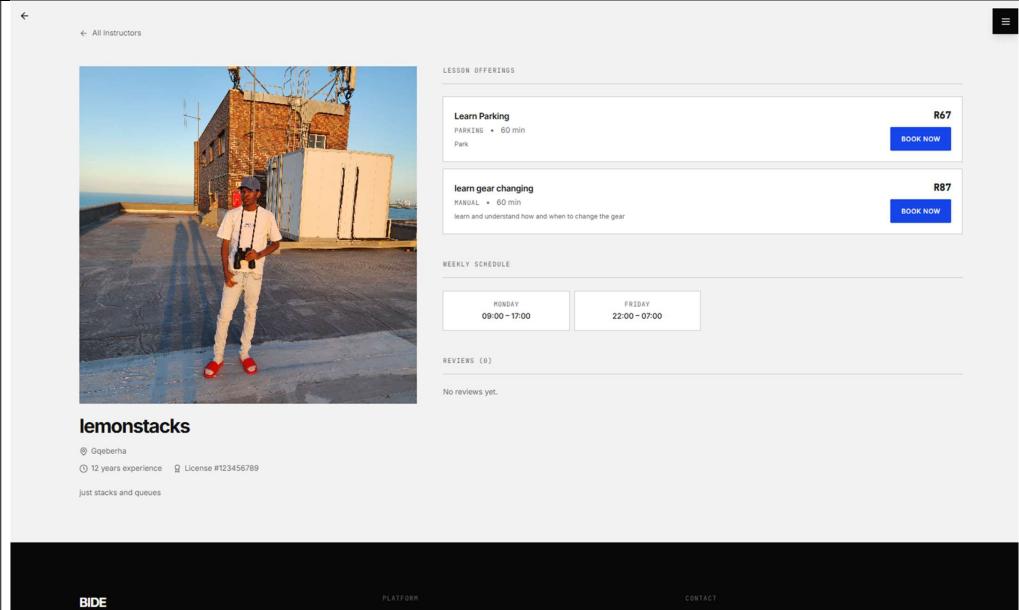
Commented [Janine20]:
Repeat tables below for every use case
Ensure numbering in each table for the flow of events start at 1 for each use case

2.4.2.5 Designed by <Hlomani Mashimbye>

Use Case ID	Use Case Name
UC05_S	View Instructor Profile
Actors	
Student	

Commented [Janine21]:
Repeat tables below for every use case
Ensure numbering in each table for the flow of events start at 1 for each use case

Pre-Conditions	Search results are available
Triggers	Student selects an instructor <i>called from UC04-S</i>
Post-Conditions	Instructor profile is displayed
Basic Flow of Events / Main Success Scenario	
The actor selects an instructor from the search results. <i>A1b3</i> - The system retrieves the instructor's profile details. - The system displays the instructor's profile. <i>3. User selects Book Now</i>	
Initial UI Design <i>4. System calls UC06-S</i>	



Final Implemented UI Design

2.4.2.6 Designed by <Lethabo Mathole>

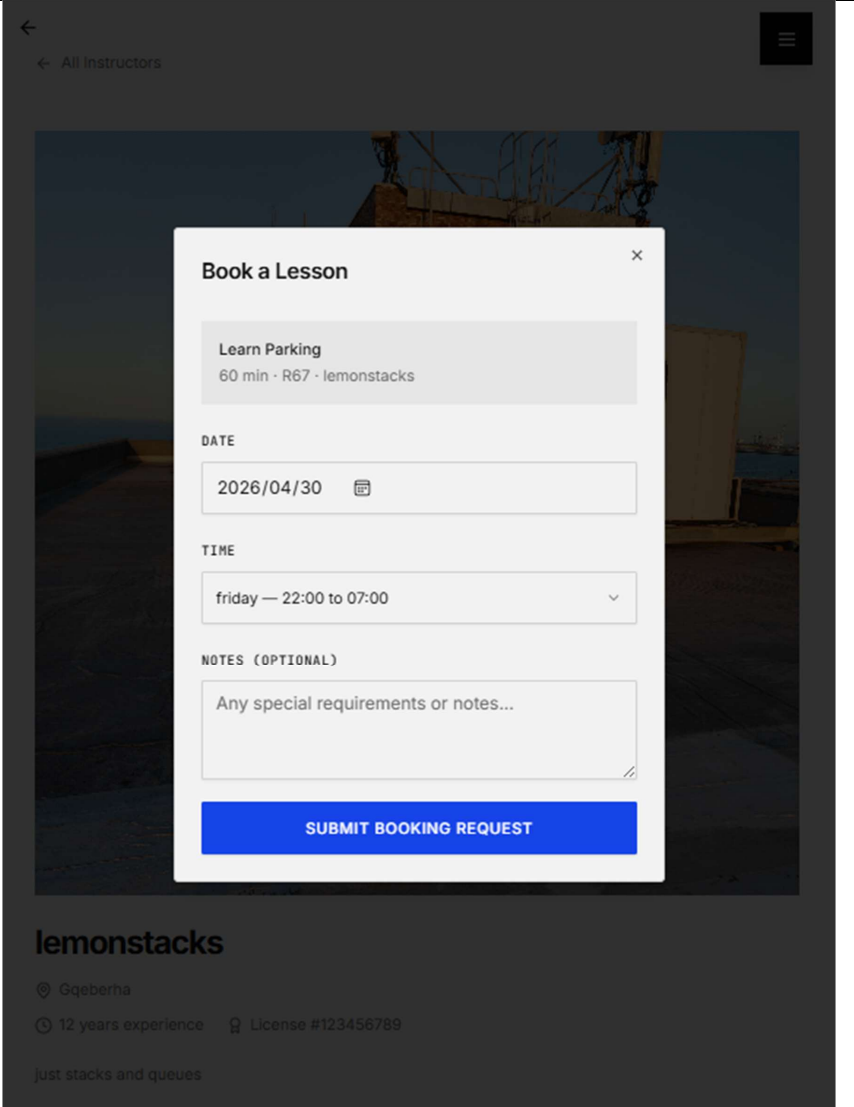
Use Case ID	Use Case Name
UC06_S	Make Booking Request
Actors	
Student	
Pre-Conditions	Student has viewed an instructor profile
Triggers	Student selects booking option <i>called from UC06-S</i>
Post-Conditions	Booking request is recorded as pending
Basic Flow of Events / Main Success Scenario	
The actor selects an instructor. <i>System displayed Booking Form</i> - The actor selects a preferred date and time. - The system checks the instructor's availability.	

→ actor clicks Submit..... button

Commented [Janine22]:
Repeat tables below for every use case
Ensure numbering in each table for the flow of events start at 1 for each use case

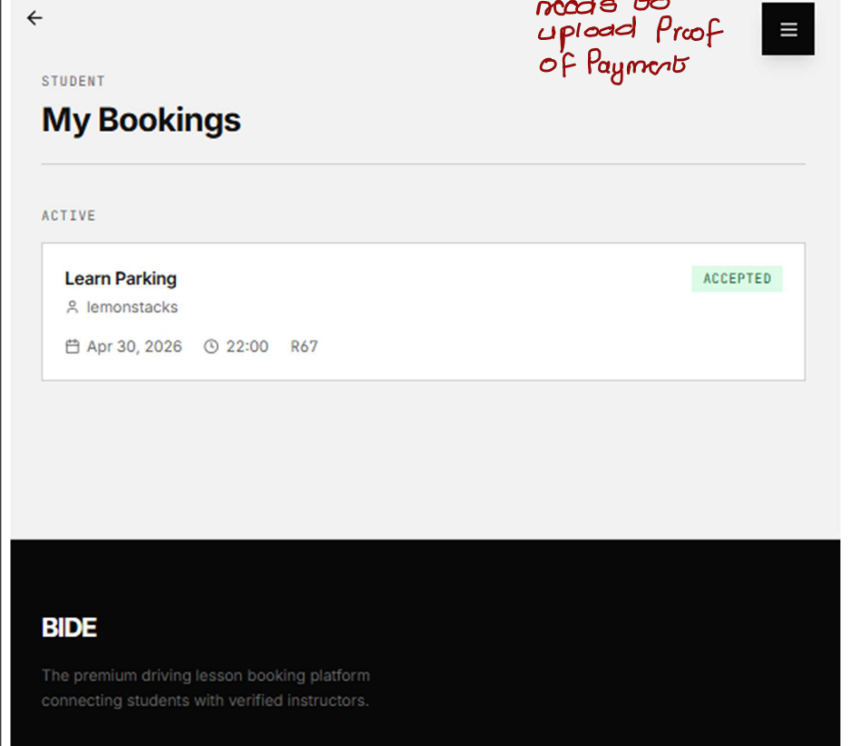
- 4. The system confirms the selected time slot is available.
- 5. The actor confirms the booking request.
- 6. The system records the booking request.
- 7. The system marks the booking as pending.
- 8. The system displays a confirmation message.

Initial UI Design



Final Implemented UI Design

2.4.2.7 Designed by <Lethabo Mathole>

Use Case ID	Use Case Name
UC07_S	View Booking Status
Actors	
Student	
Pre-Conditions	Student has bookings
Triggers	Student selects booking status option from 
Post-Conditions	Booking statuses are displayed
Basic Flow of Events / Main Success Scenario	
1. The actor selects the option to view bookings. Alt 3 2. The system retrieves all booking records. 3. The system displays booking statuses. - You need to describe Step for when user made to upload Proof of Payment	
Initial UI Design	
	
Final Implemented UI Design	

Commented [Janine23]:
Repeat tables below for every use case
Ensure numbering in each table for the flow of events start at 1 for each use case

2.4.2.8 Designed by <Lethabo Mathole>

Commented [Janine24]:
Repeat tables below for every use case
Ensure numbering in each table for the flow of events start at 1 for each use case

Use Case ID	Use Case Name
UC08_S	Upload Proof of Payment
Actors	
Student	
Pre-Conditions	A booking request exists
Triggers	Student selects upload option <i>called from UC07-S</i>
Post-Conditions	Payment proof is stored
Basic Flow of Events / Main Success Scenario	
1. The actor selects a booking. 2. The system displays booking details. 3. The actor uploads proof of payment. <i>(select Choose File)</i> 4. The system validates the uploaded file. 5. The system stores payment proof.	
Initial UI Design	
← STUDENT My Bookings ACTIVE Learn Parking ACCEPTED Apr 30, Learn Parking ACCEPTED R67 INSTRUCTOR lemonstacks DATE Apr 30, 2026 TIME 22:00 DURATION 60 min UPLOAD PAYMENT PROOF Choose File CapitecPOP.jpg Uploading... Cancel Booking <i>this does not match with UC purpose</i>	
Final Implemented UI Design	

Use Case ID	Use Case Name
UC09_S	View completed lessons
Actors	
Student	
Pre-Conditions	Completed lessons exist
Triggers	Student selects completed lessons option <i>from</i>
Post-Conditions	Completed lessons are displayed
Basic Flow of Events / Main Success Scenario	
The actor selects the option to view completed lessons. - The system retrieves completed lesson records. - The system displays the list of completed lessons	<i>A163</i> <u>Select</u> <i>Leave Review</i>
Initial UI Design	

←

STUDENT

Completed Lessons

Learn Parking

lemonstacks

📅 Apr 30, 2026

🕒 22:00

LEAVE REVIEW

BIDE

The premium driving lesson booking platform connecting students with verified instructors.

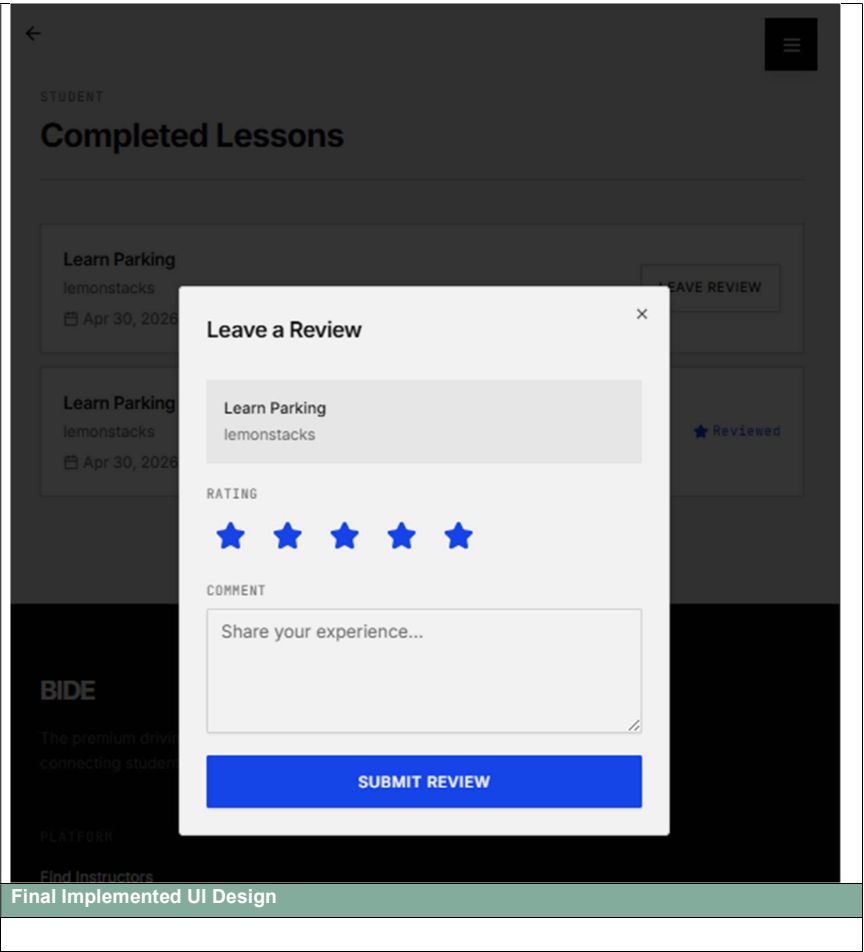
Final Implemented UI Design

2.4.2.10 Designed by <Lethabo Mathole>

Use Case ID	Use Case Name
UC10_S	Leave Review
Actors	
Student	
Pre-Conditions	Student has completed a lesson
Triggers	Student selects leave review option <i>called from UC09_S</i>
Post-Conditions	Review is saved
Basic Flow of Events / Main Success Scenario	
7. The actor selects a completed lesson. 8. The system displays the review form. 9. The actor enters a rating and comment. 10. The system validates the review details. 11. The system saves the review. 12. The system displays a confirmation message.	
Initial UI Design	

Commented [Janine25]:
Repeat tables below for every use case
Ensure numbering in each table for the flow of events start at 1 for each use case

check numbering



2.4.2.11 Designed by <Awonke Gana>

Use Case ID	Use Case Name
UC11_I	Manage Availability
Actors	
Instructor	
Pre-Conditions	Instructor account exists
Triggers	Instructor selects availability <i>from ≡</i>
Post-Conditions	Availability schedule is updated
Basic Flow of Events / Main Success Scenario	
1. The actor selects availability settings.	

Commented [Janine26]:
Repeat tables below for every use case
Ensure numbering in each table for the flow of events start at 1 for each use case

2. The system displays current availability.
3. The actor updates available time slots.
4. The system validates entries.
5. The system saves the schedule.
6. The system displays updated availability.

- need to split
edit/delete as
alternates

Initial UI Design

INSTRUCTOR

Manage Availability

DAY

friday

START

22:00

END

07:00

+ ADD

MONDAY

09:00 - 17:00

FRIDAY

22:00 - 07:00

BIDE

The premium driving lesson booking platform
connecting students with verified instructors.

Final Implemented UI Design

2.4.2.12 Designed by <Awonke Gana>

Use Case ID	Use Case Name
UC12_I	Manage lesson offerings
Actors	
Instructor	
Pre-Conditions	Instructor account exists
Triggers	Instructor selects lesson offerings <i>from ≡</i>
Post-Conditions	Lesson offerings are updated
Basic Flow of Events / Main Success Scenario	
1. The actor selects lesson offerings. 2. The system displays existing offerings. 3. The actor adds or <u>edits</u> lesson details. ✓ 4. The system validates the information. 5. The system saves the changes. 6. The system displays updated offerings. - should be an alternate - also include Delete	
Initial UI Design	
← ≡ INSTRUCTOR Lesson Offerings + ADD OFFERING Learn Parking PARKING · 60 min · R67 ✎ 🗑 learn gear changing MANUAL · 60 min · R87 ✎ 🗑 BIDE The premium driving lesson booking platform connecting students with verified instructors.	
Final Implemented UI Design	

Commented [Janine27]:
Repeat tables below for every use case
Ensure numbering in each table for the flow of events start at 1 for each use case

2.4.2.13 Designed by <Lethabo MatholeAwonke-Gana>

Use Case ID	Use Case Name
UC13_I	View Booking Requests
Actors	
Instructor	
Pre-Conditions	Booking requests exist
Triggers	Instructor selects booking requests option <i>from ☰</i>
Post-Conditions	Booking requests are displayed
Basic Flow of Events / Main Success Scenario	
The actor selects the option to view booking requests. <i>A153</i> - The system retrieves all booking requests. <i>- select Respond</i> - The system displays the booking request list. <i>- call UC14-1</i>	
Initial UI Design	

←

☰

INSTRUCTOR

Booking Requests

ACCEPTED

Learn Parking

ACCEPTED

fiemyfx

Apr 30, 2026 · 22:00 · R67

RESPOND

BIDE

The premium driving lesson booking platform connecting students with verified instructors.

Final Implemented UI Design	
-----------------------------	--

Commented [Janine28]:
Repeat tables below for every use case
Ensure numbering in each table for the flow of events start at 1 for each use case

2.4.2.14 Designed by <Awonke Gana>

Use Case ID	Use Case Name
UC14_I	Respond to Booking Request
Actors	
Instructor	
Pre-Conditions	Booking request exists
Triggers	Instructor selects a request <i>called from UC13-1</i>
Post-Conditions	Booking status is updated
Basic Flow of Events / Main Success Scenario	
The actor selects a booking request. - The system displays request details. - The actor reviews the request. - The actor accepts or rejects the request. - The system updates the booking status. - The system stores the response.	
Initial UI Design	
Final Implemented UI Design	

Commented [Janine29]:
Repeat tables below for every use case
Ensure numbering in each table for the flow of events start at 1 for each use case

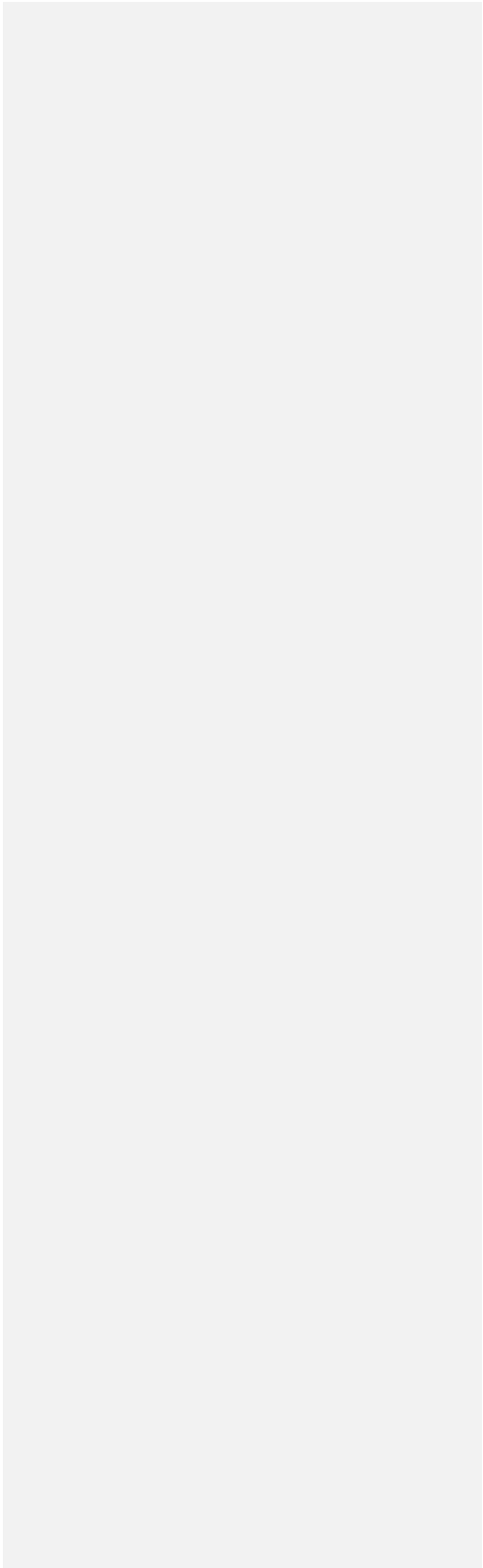
2.4.2.15 Designed by <Lethabo Mathole>

Use Case ID	Use Case Name																																			
UC15_I	Verify Payment																																			
Actors																																				
Instructor																																				
Pre-Conditions	Payment proof is uploaded																																			
Triggers	Instructor selects payment verification <i>from</i>																																			
Post-Conditions	Payment is verified or rejected																																			
Basic Flow of Events / Main Success Scenario <i>1. System displays V</i>																																				
1. The actor selects a booking with payment proof. 2. The system displays the payment details. 3. The actor reviews proof of payment. 4. The actor verifies or rejects the payment. 5. The system updates the payment status. 6. The system saves the result.																																				
Initial UI Design																																				
INSTRUCTOR Verify Payment Review and verify student payments for bookings. Pending Verification Verified Payments <table><tr><th>Booking ID</th><th>Student</th><th>Lesson</th><th>Lesson Date</th><th>Amount</th><th>Proof of Payment</th><th>Action</th></tr><tr><td>BKG1001</td><td>Lethabo Mathole lethabo.m@example.com 071 234 5678</td><td>Learn Parking (2 Hours)</td><td>30 Apr 2026 22:00</td><td>R350.00</td><td> receipt_1001.jpg Uploaded on 28 Apr 2026 14:32</td><td> ✓ Verify ✗ Reject </td></tr><tr><td>BKG1002</td><td>Aweike Gana aweikeng@example.com 072 345 6789</td><td>Highway Driving (2 Hours)</td><td>02 May 2026 10:00</td><td>R350.00</td><td> receipt_1002.jpg Uploaded on 30 Apr 2026 09:15</td><td> ✓ Verify ✗ Reject </td></tr><tr><td>BKG1003</td><td>Sipho Dlamini sipho.d@example.com 073 456 7890</td><td>Night Driving (2 Hours)</td><td>06 May 2026 18:00</td><td>R350.00</td><td> receipt_1003.jpg Uploaded on 01 May 2026 11:47</td><td> ✓ Verify ✗ Reject </td></tr><tr><td>BKG1004</td><td>Zanele Nkasi zanele.n@example.com 074 567 8901</td><td>Parallel Parking (2 Hours)</td><td>07 May 2026 14:00</td><td>R350.00</td><td> receipt_1004.jpg Uploaded on 02 May 2026 16:05</td><td> ✓ Verify ✗ Reject </td></tr></table> Showing 1 to 4 of 4 entries < 1 >		Booking ID	Student	Lesson	Lesson Date	Amount	Proof of Payment	Action	BKG1001	Lethabo Mathole lethabo.m@example.com 071 234 5678	Learn Parking (2 Hours)	30 Apr 2026 22:00	R350.00	 receipt_1001.jpg Uploaded on 28 Apr 2026 14:32	✓ Verify ✗ Reject	BKG1002	Aweike Gana aweikeng@example.com 072 345 6789	Highway Driving (2 Hours)	02 May 2026 10:00	R350.00	 receipt_1002.jpg Uploaded on 30 Apr 2026 09:15	✓ Verify ✗ Reject	BKG1003	Sipho Dlamini sipho.d@example.com 073 456 7890	Night Driving (2 Hours)	06 May 2026 18:00	R350.00	 receipt_1003.jpg Uploaded on 01 May 2026 11:47	✓ Verify ✗ Reject	BKG1004	Zanele Nkasi zanele.n@example.com 074 567 8901	Parallel Parking (2 Hours)	07 May 2026 14:00	R350.00	 receipt_1004.jpg Uploaded on 02 May 2026 16:05	✓ Verify ✗ Reject
Booking ID	Student	Lesson	Lesson Date	Amount	Proof of Payment	Action																														
BKG1001	Lethabo Mathole lethabo.m@example.com 071 234 5678	Learn Parking (2 Hours)	30 Apr 2026 22:00	R350.00	 receipt_1001.jpg Uploaded on 28 Apr 2026 14:32	✓ Verify ✗ Reject																														
BKG1002	Aweike Gana aweikeng@example.com 072 345 6789	Highway Driving (2 Hours)	02 May 2026 10:00	R350.00	 receipt_1002.jpg Uploaded on 30 Apr 2026 09:15	✓ Verify ✗ Reject																														
BKG1003	Sipho Dlamini sipho.d@example.com 073 456 7890	Night Driving (2 Hours)	06 May 2026 18:00	R350.00	 receipt_1003.jpg Uploaded on 01 May 2026 11:47	✓ Verify ✗ Reject																														
BKG1004	Zanele Nkasi zanele.n@example.com 074 567 8901	Parallel Parking (2 Hours)	07 May 2026 14:00	R350.00	 receipt_1004.jpg Uploaded on 02 May 2026 16:05	✓ Verify ✗ Reject																														
Final Implemented UI Design																																				

Commented [Janine30]:
Repeat tables below for every use case
Ensure numbering in each table for the flow of events start at 1 for each use case

from ≡

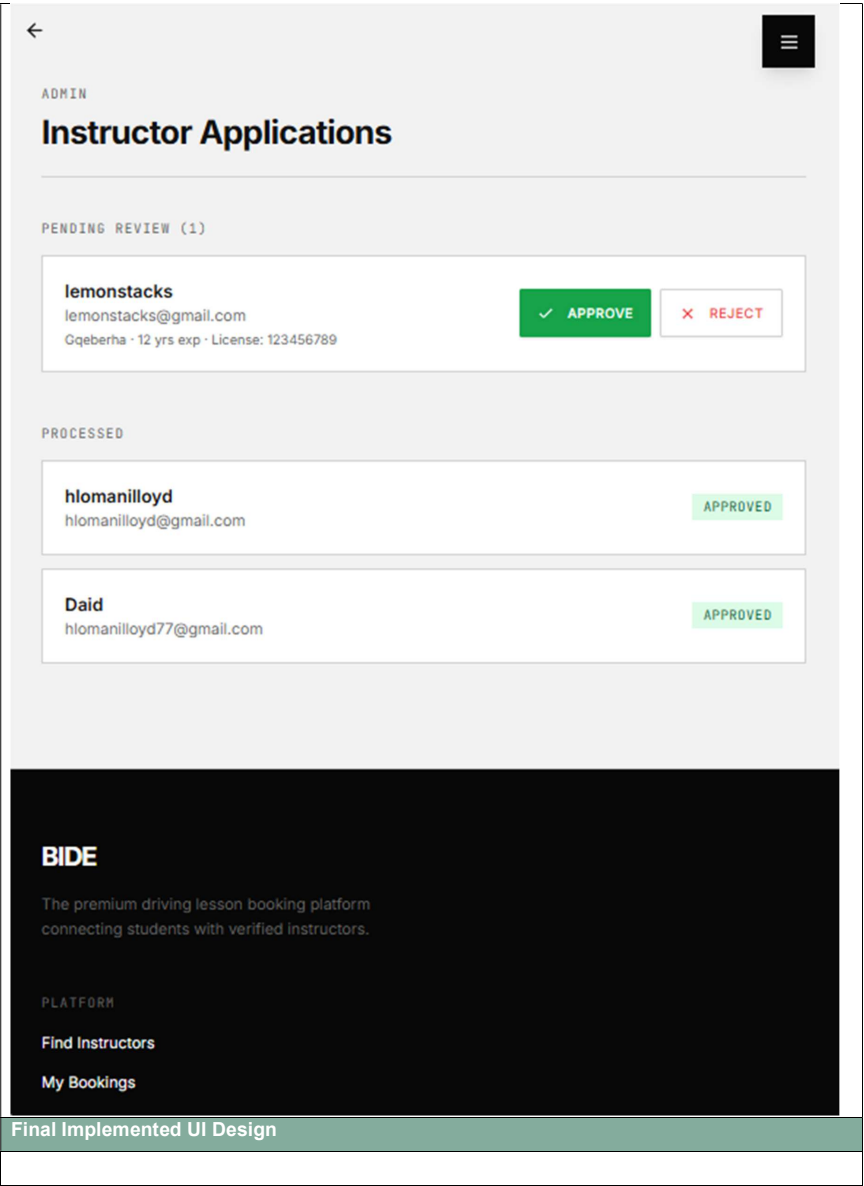
1. System displays Verify Payments UI



2.4.2.16 Designed by <Hlomani Mashimbye>

Use Case ID	Use Case Name
UC16_A	Review Instructor Application
Actors	
Admin	
Pre-Conditions	Instructor application exists
Triggers	Admin selects application <i>from ≡</i>
Post-Conditions	Application is approved or rejected
Basic Flow of Events / Main Success Scenario	
1. The actor selects an instructor application. 2. The system displays application details. 3. The actor reviews the application. 4. The actor approves or rejects the application. 5. The system updates the application status.	
Initial UI Design	

Commented [Janine31]:
Repeat tables below for every use case
Ensure numbering in each table for the flow of events start at 1 for each use case



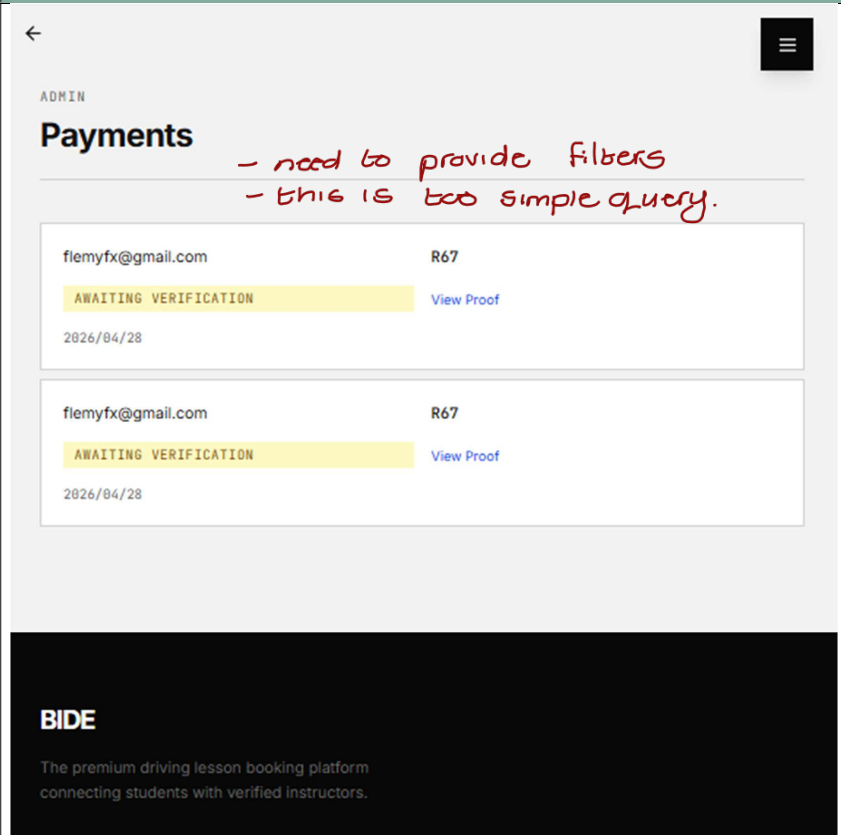
2.4.2.17 Designed by <Hlomani Mashimbye>

Use Case ID	Use Case Name
UC17_A	Manage Users

Commented [Janine32]:
Repeat tables below for every use case
Ensure numbering in each table for the flow of events start at 1 for each use case

Actors																									
Admin																									
Pre-Conditions	User accounts exist																								
Triggers	Admin selects user management <i>from ...</i>																								
Post-Conditions	User records are updated																								
Basic Flow of Events / Main Success Scenario																									
1. The actor selects user management. 2. The system displays all user accounts. 3. The actor selects a user account. 4. The actor performs an action such as editing or <u>deactivating</u>. <i>Where is the option on the UI</i> 5. The system updates the user's record. 6. The system saves the changes.																									
Initial UI Design																									
ADMIN Manage Users <table><tr><th>NAME</th><th>EMAIL</th><th>ROLE</th><th></th></tr><tr><td>flamyfx</td><td>flamyfx@gmail.com</td><td> Instructor </td><td>4/28/2026</td></tr><tr><td>lemonstacks</td><td>lemonstacks@gmail.com</td><td> Instructor </td><td>4/28/2026</td></tr><tr><td>nomorechange25</td><td>nomorechange25@gmail.com</td><td> Instructor </td><td>4/27/2026</td></tr><tr><td>hlomanilloyd</td><td>hlomanilloyd@gmail.com</td><td> Student </td><td>4/27/2026</td></tr><tr><td>Hlomanil Lloyd</td><td>hlomanilloyd77@gmail.com</td><td> Admin </td><td>4/7/2026</td></tr></table> <i>can Admin thus change Role? do not think it is a good idea</i>		NAME	EMAIL	ROLE		flamyfx	flamyfx@gmail.com	Instructor	4/28/2026	lemonstacks	lemonstacks@gmail.com	Instructor	4/28/2026	nomorechange25	nomorechange25@gmail.com	Instructor	4/27/2026	hlomanilloyd	hlomanilloyd@gmail.com	Student	4/27/2026	Hlomanil Lloyd	hlomanilloyd77@gmail.com	Admin	4/7/2026
NAME	EMAIL	ROLE																							
flamyfx	flamyfx@gmail.com	Instructor	4/28/2026																						
lemonstacks	lemonstacks@gmail.com	Instructor	4/28/2026																						
nomorechange25	nomorechange25@gmail.com	Instructor	4/27/2026																						
hlomanilloyd	hlomanilloyd@gmail.com	Student	4/27/2026																						
Hlomanil Lloyd	hlomanilloyd77@gmail.com	Admin	4/7/2026																						
Final Implemented UI Design																									

2.4.2.18 Designed by <View PaymentLethabo Matholes>

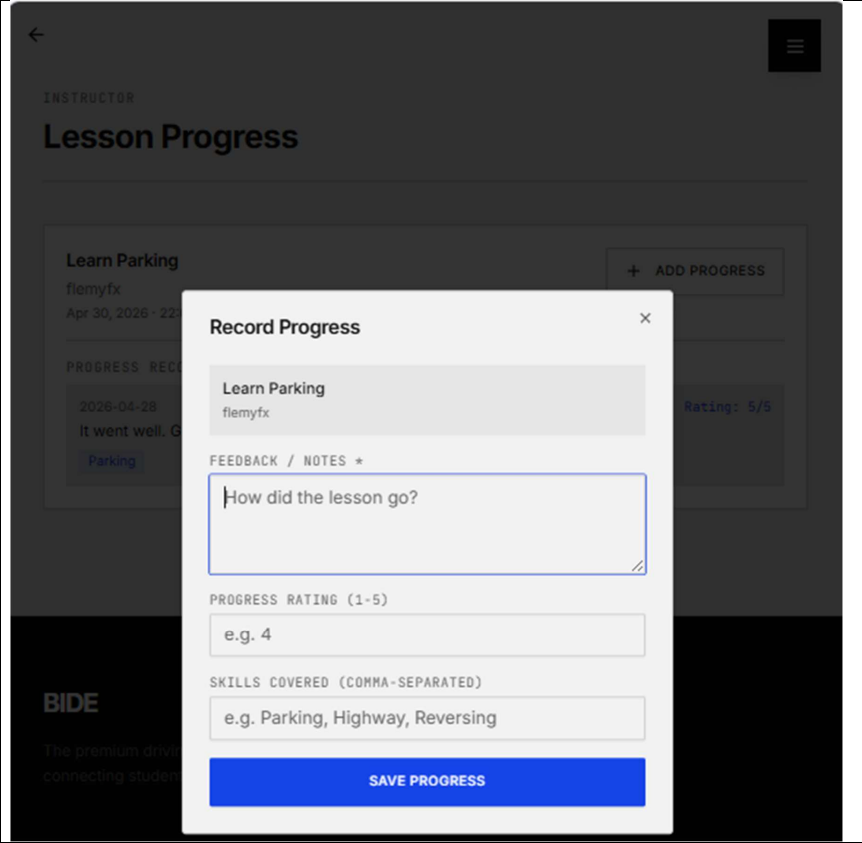
Use Case ID	Use Case Name
UC18_A	View Payments
Actors	
Admin	
Pre-Conditions	Payments records exist
Triggers	Admin selects payments option <i>from</i>
Post-Conditions	Payment details are displayed
Basic Flow of Events / Main Success Scenario	
1. The actor selects the option to view payments. 2. The system retrieves payment records. 3. The system displays payment details.	
Initial UI Design	
 The mockup shows an 'ADMIN' interface with a 'Payments' section. It contains two identical payment record cards. Each card displays the email 'flemyfx@gmail.com', the amount 'R67', a yellow 'AWAITING VERIFICATION' status bar, a 'View Proof' link, and the date '2026/04/28'. Handwritten red notes are present: '- need to provide fibers' and '- this is too simple query.' at the top of the cards. At the bottom of the mockup is a black footer with the text 'BIDE' and 'The premium driving lesson booking platform connecting students with verified instructors.'	
Final Implemented UI Design	

Commented [Janine33]:
Repeat tables below for every use case
Ensure numbering in each table for the flow of events start at 1 for each use case

2.4.2.20 Designed by <Awonke Gana>

Use Case ID	Use Case Name
UC20_I	Provide Lesson Feedback
Actors	
Instructor	
Pre-Conditions	A lesson session associated with a specific booking has been initiated or completed.
Triggers	The Instructor selects the option to add progress. <i>where?</i>
Post-Conditions	Feedback is stored, displayed, and linked to the student's lesson history.
Basic Flow of Events / Main Success Scenario	
1. The instructor accesses the lesson progress tab 2. The system will display the lesson progress for each lesson. 3. The instructor enters qualitative feedback and performance ratings. 4. The system saves the feedback record 5. The system confirms the feedback has been successfully logged. <i>1st select Student, then Lesson</i>	
Initial UI Design	

Commented [Janine35]:
Repeat tables below for every use case
Ensure numbering in each table for the flow of events start at 1 for each use case



Final Implemented UI Design

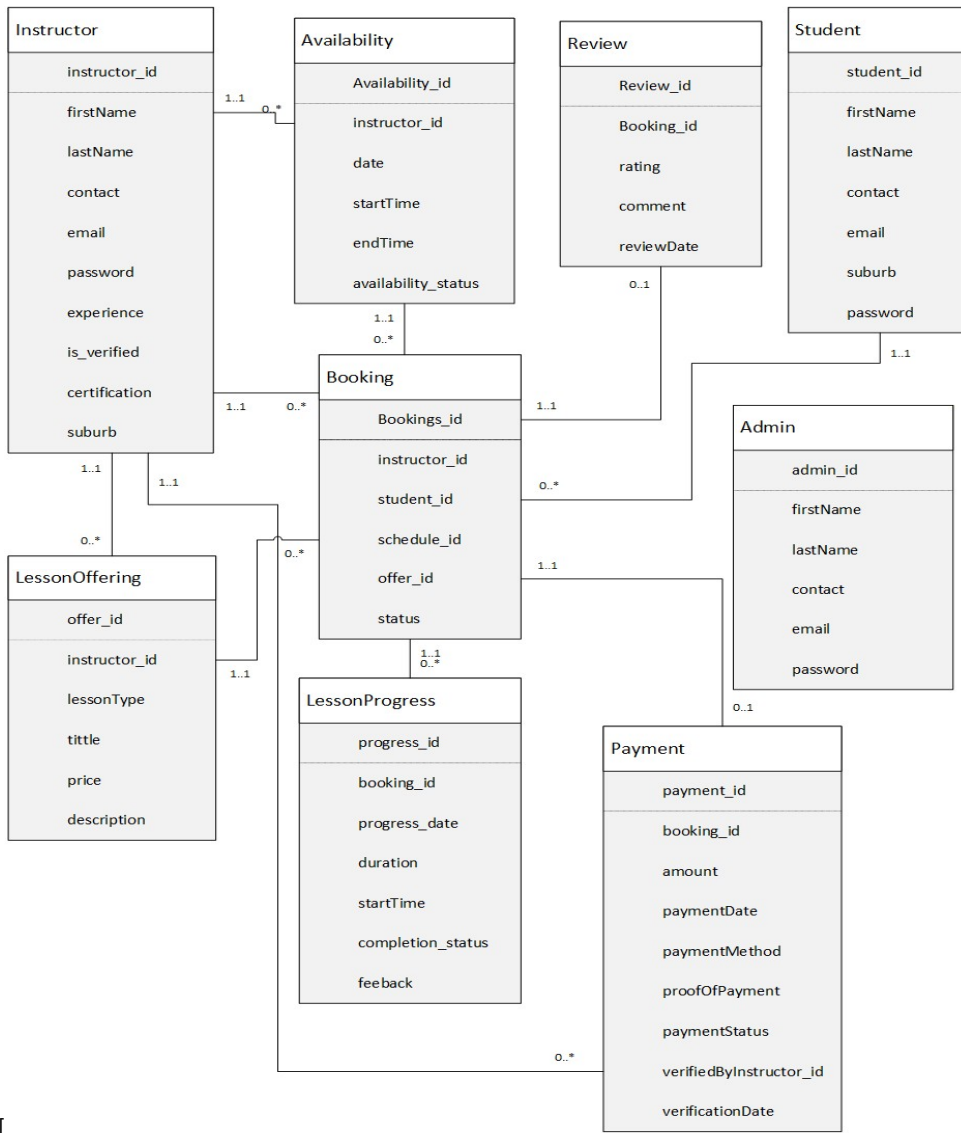
3 DATA REQUIREMENTS

3.1 List of data and attributes

Entity	Entity Attributes
Student	studentId: int, licenseNo: String, phone: String, dateOfBirth: Date
Instructor	instructorId: int, licenseNo: String, bio: String, rating: double
Admin	adminId: int, role: String, accessLevel: int
Booking	bookingId: int, date: Date, time: Time, location: String, status: String, duration: int
LessonOffering	offeringId: int, type: String, price: double, duration: int, description: String
Availability	availabilityId: int, dayOfWeek: String, startTime: Time, endTime: Time
Payment	paymentId: int, amount: double, proofFile: String, status: String, verifiedAt: DateTime
Review	reviewId: int, rating: int, comment: String, createdAt: DateTime, bookingId: int
InstructorApplication	applicationId: int, submittedAt: DateTime, status: String, documents: String, reviewedBy: int

Commented [Janine37]:
Data requirements can be modeled with a Data Model. This is a list of relevant business entities about which we need to capture and store data

3.2 Domain Class Diagram



Commented [Janine38]:
 Draw a domain class diagram using UML notation based on the data you have identified in 4.1
 Multiplicity must be shown
 No need to show PK and FK at this stage
 Show inheritance where applicable

- This is not Domain Class
 but Implementation Ready

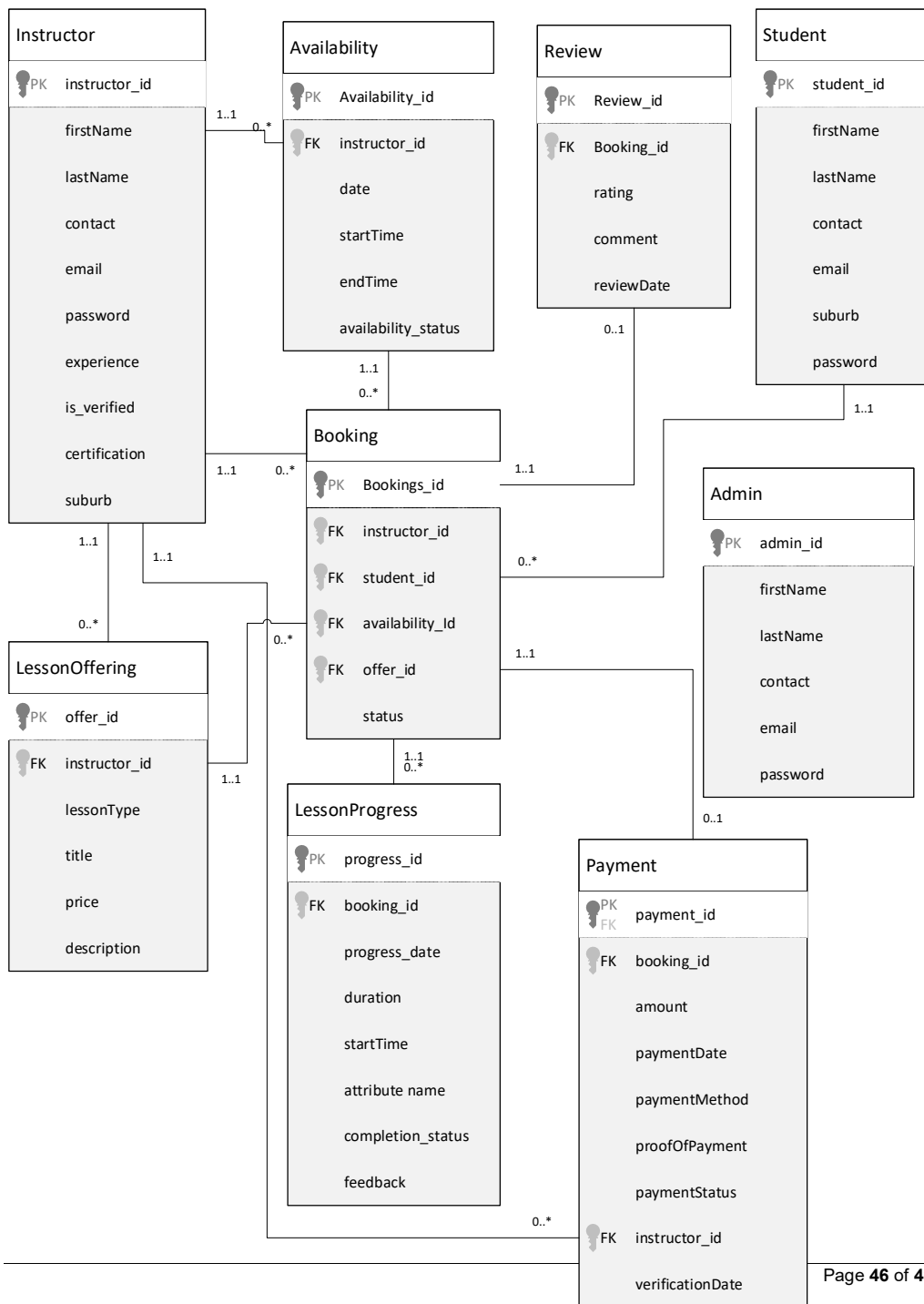
- Need to show Inheritance
 + Association classes

3.3 Draft **Implementation** Ready Class Diagram

Commented [NJ](CS39): Draw an implementation ready Entity Relationship Diagram (ERD) using UML notation

Refer to your WRDBV notes for an explanation of this diagram

Only show the Pks and FKs on the diagram



3.4 Implemented database structure

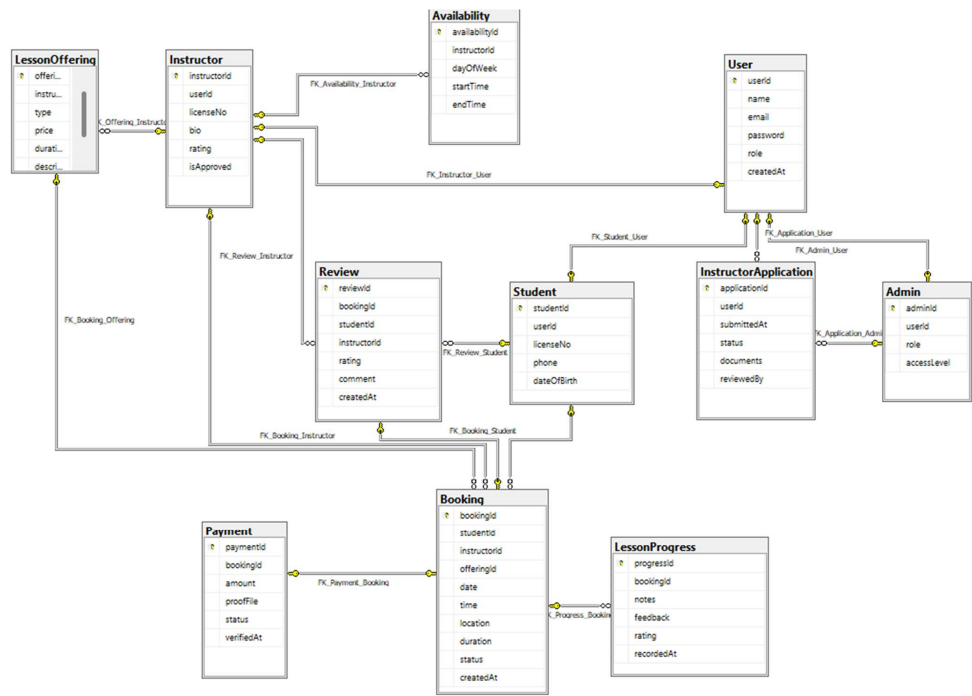
We have used <SQL Server> to create our database in.

The name of the database for our project is <BIDE>

Commented [NJ](CS40): Replace with the DBMS name you have used i.e. MS-Access, MySQL, SQLite

Commented [NJ](CS41): What did you call your database?

[Replace this text with screenshot of the relationship view of the implemented database]



4 TECHNICAL REQUIREMENTS

4.1 System Environment

4.1.1.1 Operating Environment

The system will be developed and executed as a **web-based application**. It can run on any modern operating system that supports web browsers, including:

- Windows 10 / Windows 11
- macOS
- Linux

Users will access the system through a web browser such as:

- Google Chrome (recommended)
- Microsoft Edge
- Mozilla Firefox

4.1.1.2 Development Tools and Technologies

The system will be developed using the following tools and technologies:

- **Frontend:** HTML5, CSS3, JavaScript
- **Backend:** ASP.NET MVC
- **Programming Language:** C#
- **Database:** SQL Server
- **IDE:** Visual Studio 2022

4.1.1.3 Additional Components (DLLs / Libraries)

The following libraries and frameworks may be used:

- .NET Framework / .NET Core libraries
- ASP.NET MVC libraries
- Entity Framework (for database operations)

These libraries are automatically managed within Visual Studio and do not require manual installation by the end user.

4.1.1.4 Recommended Screen Resolution

For optimal user experience, the recommended screen resolution is:

1366 x 768 or higher

For best display:

1920 x 1080 (Full HD)

4.1.1.5 Other Requirements

- Internet connection required to access the system
- An internet
- Basic computer literacy for users

Commented [NJ](CS42): Any prerequisites for running system

Indicate the environment that the system will run on i.e. operating system

Specify the development tools used and versions

DLL files that you might have used at home

Recommended screen resolution

- User must have JavaScript enabled

4.2 Start-up Instructions

4.3 Logins & Passwords

Commented [NJ](CS43): Clear instructions relating to installing or / and running your system

The name of the file to run and where (in which folder) it can be found

Commented [NJ](CS44): All Admin login & passwords (if applicable)